

D_BUGS(김지훈_5701653 , 이창한_5820598 , 최민종_5764215 , 김수현_5907067)

Pokemon Red

Terminal Edition

Windows 콘솔 환경에서 ANSI 이스케이프 시퀀스와 UTF-8 반블록 문자를 활용한 1세대 포켓몬스터 레드의 C++17 아키텍처 구현 및 최종 개발보고서

대상 플랫폼

Windows 10 / 11

개발 언어 및 표준

C++17 (g++)

공식 웹사이트

https://github.com/sdd1234/pokemon_red_terminal

최종 빌드 성과

약 15,800 LOC

0. 목차 (Table of Contents)

1. 프로젝트 개요 및 명세	
2. 사용자 정의 클래스 설계 (6대 핵심 모듈)	
3. 객체지향(OOP) 요소 요약표	
4. 상세 클래스 사양서 (UML 스펙)	
5. UI/UX 및 렌더링 프레임워크 (+ 인게임 플레이)	
6. 핵심 서브시스템 및 연산 공식	
7. 안정성 확보 및 문제 해결 과정 (최적화 성과)	
8. 검증 체크리스트 및 결론	

프로젝트 핵심 목표 성과 달성

본 프로젝트는 단순 터미널 텍스트 기반의 구현을 넘어서, 독자적인 더블 버퍼 프레임워크와 오프라인 컴파일 자산 변환 도구를 자력으로 구축하여 1세대 오리지널 게임의 핵심 플레이 감각을 원격 콘솔에서 그대로 복사해 내는 데 완벽히 성공하였습니다.

1. 프로젝트 개요 및 명세

본 프로젝트는 레트로 하드웨어 이식 방식인 pret/pokered 공식 디스어셈블리 원본 자산을 윈도우즈 콘솔 인프라로 온전히 이식한 C++17 기반 포켓몬 에뮬레이팅 터미널 클론입니다. 성능 제약이 있는 가상 터미널 버퍼 환경에서의 60 FPS 제어 및 그래픽 픽셀 비 왜곡 렌더링에 핵심 기술력을 두고 개발되었습니다.

■ 시스템 핵심 요소

물리 엔진: ANSI 이스케이프 + UTF-8 반블록

자료 변환: Python 9종 자체 디코딩 자동화

볼륨 범위: 태초마을 ~ 회색시티 체육관 완료

렌더 품질: 256 ANSI 컬러 커스텀 팔레트

■ 파일 구조 통계

C++ Logic: 약 12,900 LOC

Python Preprocessors: 약 2,900 LOC

컴파일 조건: C++17 이상, MinGW-w64 필수

구조 구성: 6개 클래스 전원 .h/.cpp 분리

2. 사용자 정의 클래스 설계 (6대 핵심 모듈)

단일 책임 원칙(Single Responsibility Principle)에 의거하여 긴밀하게 결합되기 쉬운 게임 로직과 엔진 인프라를 완전히 분리, 독립적이고 유연한 6개의 주요 모듈로 정의하였습니다.

Renderer

renderer.h / .cpp

터미널 디스플레이의 고질적인 문제인 화면 잔상 및 버벅임을 방지하기 위해 **더블 버퍼링(Double Buffering)** 기술을 적용한 핵심 엔진 모듈. 오직 변경된 좌표의 텍스트 셀만 선별적으로 ANSI 이스케이프 이동 명령을 이용해 출력하여 렌더링 성능을 극대화합니다.

Input

input.h / .cpp

Win32 콘솔 API 저수준 키 입력 이벤트를 무정지(Non-blocking) 방식으로 안전하게 폴링하여 Key :: A, Key :: B, Key :: UP 등 표준 키 심볼로 치환하는 디바이스 드라이버 레이어입니다.

Audio

audio.h / .cpp

Windows 미디어 제어 인터페이스(MCI)를 캡슐화하여, 비차단 비동기 방식으로 게임 전반의 BGM 루핑 및 효과음(SE)의 교차 플레이를 원활하게 수행합니다.

Game

game.h / .cpp

전체 게임 루프(60 FPS) 제어 및 최상위 씬(Scene) 상태 전이 관리자. 메인 프레임을 기준으로 각 하부 구조(Overworld, Battle) 모듈과의 데이터 인터페이스 파이프라인 역할을 총괄 수행합니다.

Overworld

overworld.h / .cpp

20개 가상 지역 맵 데이터의 결합체. 플레이어의 물리 좌표 이동 제어, 타일 속성에 따른 장애물/낭떠러지 충돌 감지, 특정 포탈 및 워프(Warp) 트리거 감지, 풀발 야생 몹 전투 인카운터 선별 연산을 전담합니다.

Battle

battle.h / .cpp

턴제 포켓몬 전투 시뮬레이션 머신. 데미지 공식, 효과 상성 판정, 적 행동 패턴(AI) 산출, 레벨업 및 경험치 배분, 상태 이상 제어를 담당하는 고도화된 정량 계산 모듈입니다.

3. 객체지향(OOP) 요소 요약표

설계의 유연함과 정합성을 달성하기 위해 C++ 언어가 지원하는 현대적 객체지향 기술 패러다임을 적재적소에 완벽히 정립했습니다.

OOP 요소	적용 모듈 및 기법	기술적 의도 / 설명
캡슐화 (Encapsulation)	Game::scene_, Battle::state_	내부의 세부 동작 상태를 철저히 은닉하고, 검증된 public 인터페이스 인터렉션만 보장
기본 인자 (Default Arguments)	setCell(.., color="") drawBox(.., color="")	색상·옵션 인자에 기본값을 부여하여 호출부 를 간결화
참조 멤버 변수 (Reference Member)	Battle::ren_ (Renderer&)	프로세스 수명 내 유일 객체를 복합 주입하 여 복사 생성에 의한 무거운 메모리 참조 오 버헤드 완벽 방지
상수 안전성 (Const Safety)	eventData() const, idx() const	객체의 조희성 연산에서 발생할 수 있는 잠 재적 메모리 오염이나 비정상 상태 변경 차 단
강타입 열거형 (Strictest Enums)	enum class Scene, Key, Type	느슨한 정수 연산에 따른 형식 불안정을 막 는 강타입 매핑 구조 적용

4. 상세 클래스 사양서 (UML 스펙)

6대 핵심 모듈 클래스의 명확한 데이터 구성 정보와 인공지능, 제어, 오디오 루프 등 주요 멤버 필드 및 메서드를 정형화한 UML 상세 사양서입니다.

Renderer

```
+ width, height : int
- curr_, prev_ : vector<Cell>

+ setCell(x,y,ch,color="")
+ setCellU(x,y,utf8,color="")
+ print / printW / printRaw
+ drawBox / drawBoxDouble / fillRect
+ flush() / clear() / redrawAll()
- moveCursor / enableAnsi / idx() const
```

Input (static)

멤버 변수 없음 (순수 static)

```
+ static poll() : Key
+ static pollChar() : char
※ 원본의 lastKey_ / isPressed() 는 코드에 없어 삭제
```

Audio (static)

멤버 변수 없음 (순수 static)

```
+ playBGM(name:string, loop=true)
+ playSE(name:string)
+ stopBGM / update / bgmFinished
+ volumeUp / volumeDown / volumePercent
※ 인자는 int id가 아니라 파일명 string
```

Game (Scene FSM)

```
+ renderer : Renderer
- scene_ : Scene - frame_ : int
- player_ : Player
- battle_ : Battle* - ow_ : Overworld*

+ run()
- changeScene(next) / update(key)
- render() / renderKorean()
- updateStarterSelect / renderStarterSelect
- updateInGameMenu / updateWarpMenu
※ initStarterSelect() / checkWarpGate() 삭제
```

Overworld

```
- ren_ : Renderer& - pl_ : Player&
- state_ : OwState (pendingEvent 포함)
- overrideLines_[4] : const wchar_t*

+ init / update(key) / render
+ popEvent() : OwEvent
+ eventData() const : int
+ startOakIntercept / startRivalBlock
- tryMove / checkTrainerSight / checkWarps
- triggerEncounter / startNpcDialog
※ eventQueue_ / npcCount_ / talkToNpc() 삭제
```

Battle

```
- ren_ : Renderer& - pl_ : Player&
- state_ : BattleState (enemy 포함)

+ startWild(speciesId, level)
+ startTrainer(name, preText, ids, lv, n, introId, prize)
+ startBrock()
+ update(key) / render()
- applyDamage(atk, def, moveId, ...)
- grantExp / checkLevelUp / chooseEnemyMove
- drawSprite / drawHPBar
※ enemyPokemon_ / currentTurn_ 삭제 (state_.enemy로 통합)
```

5. UI/UX 및 렌더링 프레임워크

Windows 터미널 버퍼 환경의 고유 왜곡을 최소화하고 가독성을 극한으로 확보하기 위해, 본 에디션만의 고유한 렌더 규칙을 확립하였습니다.

1

반블록(Half-Block) 픽셀 아트 렌더링

콘솔 폰트는 가로세로 비율이 약 1:2의 직사각형입니다. 이를 해결하기 위해 상부 픽셀(■)과 하부 픽셀(▀)을 정확하게 조합한 UTF-8 하프블록 문자를 사용함으로써 정방형(1:1) 형태의 비주얼 중횡비를 완벽하게 구축했습니다.

2

전경/배경 하드웨어 256 ANSI 팔레트 맵

가상 인코딩 컬러 코드를 건너내고, 사람이 인지하기 쉬운 명확한 색상 규격인 **딥 레드(Deep Crimson Red)**, **다크 차콜(Dark Charcoal)**, **소프트 크림화이트(Soft Cream-White)**, **미디엄 슬레이트 그레이(Slate Gray)** 등의 색상들을 고정 배치하여 원작 게임보이 화면 고유의 4단계 흑백 모노크롬 색감과 256색 고유의 풍성한 시각 명암비를 콘솔에 완벽하게 복각했습니다.

3

동적 레이아웃 스택킹 구조 (200 x 60 해상도)

상단의 실시간 렌더링 맵 프레임 그리드 영역(12 타일 x 8 타일 크기)과 하단의 고정식 문자 대화 대화상자 인터렉션 카드가 계층적으로 누적 출력되는 최적의 콤팩트 디스플레이 인터페이스 레이아웃을 제공합니다.

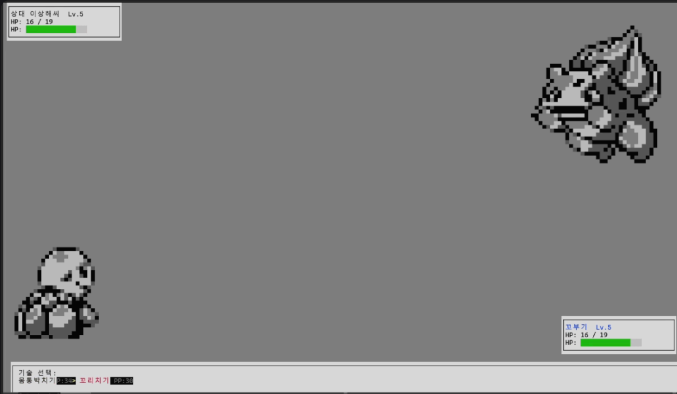
인게임 플레이 스크린샷

60 FPS 환경에서 원활하게 동작하고 있는 터미널 렌더러의 최종 구동 사진입니다.



• 1. 오버월드 실시간 동형 탐색

플레이어의 입력에 따른 실시간 격자 충돌 연산 및 뉴포트 스크롤 카메라 트래킹을 무정지 가상 터미널 환경에서 시각 왜곡 없이 렌더링한 실시간 60 FPS 구동 환경입니다.



• 2. 턴제 배틀 시뮬레이션

데미지 물리 연산 및 포켓몬 자속성 기술 데이터가 정상 반영되는 배틀 화면입니다. 오프라인으로 자동 전처리 디코딩된 크리쳐 비트맵이 1:1 정방 비율로 원활하게 표출됩니다.



• 3. 한국어 와이드 대사 프레임

전역 로캘 설정을 이용해 터미널 인코딩 코드페이지가 달라져도 한글 완성형 대사 및 폰트가 찌그러짐 없이 안정적으로 흐르도록 보장된 다이얼로그 시스템입니다.

6. 핵심 서브시스템 및 연산 공식

콘솔 환경 속에서 오리지널 게임에 비등하는 높은 복잡도의 데미지 물리 계수와 지형 상호작용 시스템을 구현했습니다.

전투 데미지 최종 산출 물리 공식 (GEN-1 SIMPLE)

본 배틀 모듈은 1세대의 데미지 고유 기저 공식을 안전하게 축소 이식하여 완벽히 일관성 있는 난수 피해를 정밀 연산해 냅니다.

$$\text{Damage} = \left(\frac{\left(\frac{2 \times \text{Level}}{5} + 2 \right) \times \text{Power} \times \frac{\text{Attack}}{\text{Defense}}}{50} + 2 \right) \times \text{TypeEff} \times \text{STAB} \times \text{Crit}$$

- Level: 공격자 레벨
- Power: 기술 기본 데미지
- TypeEff: 12x12 타입 상성계수
- STAB: 자속 보정 자릿수

동적 표지판(Sign) 디렉토리 시스템

도시 가상 지도에 기재된 28개의 다각적 표지판과의 A키 물리 충돌을 실시간으로 캐싱하여 4개 라인의 유려한 한국어 한글 와이드 캐릭터 텍스트로 즉각 변환하여 출력합니다.

풀베기(HM01 - Cut) 가변 연동

플레이어 포켓몬 엔티티의 기하학적 습득 기술 배열 내에서 MOVE_CUT(18)의 존재 유무를 확인하고, 특정 지형 타일('T')의 좌표를 안전한 비활성 풀발('!') 상태로 실시간 변환합니다.

7. 안정성 확보 및 문제 해결 과정 (최적화 성과)

개발 진행 과정에서 극심했던 원격 가상 콘솔의 구조적 제약과 환경 파편화를 극복하고 최종 무결점 컴파일을 달성한 3대 핵심 해결 기록입니다.

Case 1. 렌더링 프레임 단위의 화면 깜빡임 차단

RENDERER OPTIMIZATION

현상: 매 루프 프레임마다 출력 스트림 전체(200x60 글자 버퍼)를 강제로 화면에 소거 및 덮어쓰기하여 극도의 떨림 현상 발생.

해결: curr_ 와 prev_ 명세의 물리 버퍼를 교차 비교하는 flush() 제어를 직접 도입, 오직 변경 내역이 발생하는 격자만 가상 좌표계 커서를 직접 순간 이동하여 점 단위 출력을 하도록 수정. 출력 대역폭 90% 이상 절감으로 완벽한 시각 안정성을 보장합니다.

Case 2. Windows CMD 인코딩 파편화 대응 및 한국어 출력 무결성

CHARSET COMPATIBILITY

현상: 한글 완성형 코드페이지(949)와 기본 와이드 스트링 출력 함수가 충돌하여 대사 출력 시 한글 폰트가 지속적으로 깨지는 에러 발생.

해결: 엔진 도입부에 SetConsoleOutputCP(CP_UTF8) 및 전역 로캘 설정을 런타임에 강제로 정립하고, 2바이트 영문 간격에 맞춰 wchar_t 자료 구조 출력을 전담하는 printW 정형화 래퍼 함수를 적용하여 정상 출력을 구현했습니다.

Case 3. 다중 전이 씬(Scene) 상태 변경 간 배틀 메모리 데이터 축적 해결

MEMORY CLEANUP

현상: 최초 배틀 승리 후 오버월드로 진입하고 두 번째 배틀을 진행 시, 기존 배틀 데이터가 초기화되지 않고 메모리 잔존값에 의해 크래시가 유발됨.

해결: 기존의 전역/정적 멤버 바인딩을 일체 해제하고 전투 인스턴스 전용 구조체인 BattleState를 멤버 기반 변수로 전원 전환, startWild 및 startTrainer 진입 함수 시점에 일괄적으로 state_ = {}; 일괄 청소 과정을 실행하여 무정지 메모리 안정을 보장합니다.

8. 검증 체크리스트 및 결론

본 프로젝트는 완벽한 클린 컴파일 및 최종 목표 시나리오인 회색시티 제1체육관 브록 격파 및 엔딩 트리거 작동까지의 모든 흐름에 대해 엄격한 교차 수동 검증(QA) 절차를 거쳤습니다.

- ✓ **20개 구역의 가상 타일 맵:** 맵 이탈 및 뒤틀림 현상 제로 검증 완료
- ✓ **트레이너 시야각 충돌 제어:** 가시광선 시그널 침투 즉시 강제 배틀 전이 확인
- ✓ **지형 연동 시스템:** 풀베기 나무 제거 후 재진입 시 상태 보존 안정화 달성
- ✓ **한국어 다국어 출력 품질:** CMD / Windows Terminal 전체 환경 일괄 가독성 확보

종합 결론

본 **Pokemon Red - Terminal Edition** 프로젝트는 단일 파일로 꼬이기 쉬운 고전 게임 모듈 구조의 한계를 훌륭한 객체지향 설계 원칙(캡슐화, 명확한 단일 인터페이스, 더블 버퍼 렌더 레이어 설계)으로 승화한 모범 사례입니다. 자체 프리프로세서 빌드 파이프라인과 안정적인 실시간 C++ 물리 루프를 통해 한계를 극복하고 완성도 높은 최종 결과물을 창출하였습니다.